DWEC · Actividad C · 90 minutos

Club de videojuegos

El club lleva un registro de sus partidas y quiere consultar cuánto se juega y qué puntuaciones se consiguen. Vas a completar unas funciones de JavaScript para trabajar con esos datos. El archivo de partida ya está preparado: tu trabajo consiste en resolver los apartados señalados y comprobar que funcionan.

Nombre: _____ Grupo: _____

Cómo vamos a trabajar

Abre actividad_c_videojuegos_90min.js y ejecútalo. Puedes hacerlo desde una terminal con el siguiente comando o pegar el archivo completo en la consola del navegador.

```
node actividad_c_videojuegos_90min.js
```

Encontrarás nueve marcas TODO repartidas en seis apartados. Sustituye cada línea throw new Error("PENDIENTE...") por el código que se pide. Conserva el resto del código de partida. Vuelve a ejecutar al terminar cada apartado: el comprobador muestra OK, PENDIENTE o FALLO, junto al caso que está revisando.

Apartado	Tiempo orientativo
1 Tipos y función flecha, incluida la puesta en marcha	8 min
2 Constructor, método, parámetros por defecto y rest	15 min
3 Copias con spread y Object.assign	10 min
4 Fechas y timestamps	10 min
5 Selección con filter y parámetros opcionales	15 min
6 Suma y agrupación con reduce	17 min
Revisión de resultados y entrega	15 min
Total de la sesión	90 min

Los datos que ya tienes

partidas es un array de objetos. Cada objeto incluye id, juego, genero, minutos, puntos, fecha y etiquetas. Los nombres de los juegos son inventados. Esta tabla te ayudará a comprobar los resultados sin tener que leer todo el array cada vez.

id	Juego	Género	Minutos	Puntos
1	Órbita Azul	aventura	45	1200
2	Ciudad Pixel	plataformas	30	800
3	Rally Solar	carreras	20	600
4	Órbita Azul	aventura	60	1800
5	Bosque Run	plataformas	25	950
6	Laboratorio X	puzles	40	1500
7	Rally Solar	carreras	35	1100

1 Tipos y funciones flecha

Empieza leyendo los console.log del apartado 1. Antes de ejecutarlos, apunta en un comentario qué crees que devuelve typeof en cada caso. Fíjate especialmente en undefined, null y los arrays.

```
typeof 1200;       // "number"
typeof "1200";     // "string"
typeof undefined;  // "undefined"
typeof null;       // "object"
typeof [];         // "object"
Array.isArray([]); // true
```

undefined aparece, por ejemplo, cuando un parámetro no recibe argumento. null es un valor que se asigna expresamente para indicar ausencia. typeof null devuelve "object" por una particularidad histórica de JavaScript; para detectar null usa valor === null. Las funciones dan "function" y los booleanos, "boolean".

Completa el TODO 1 de esNumeroValido. Mantén la función flecha y devuelve true solo si numero es un number finito mayor o igual que cero. Usa typeof, Number.isFinite y una comparación. "1200", NaN, Infinity, null y undefined deben devolver false. El número 0 sí es válido.

2 Crear una partida y darle un método

En este apartado vas a construir una versión sencilla de una partida con juego, minutos, puntos y etiquetas. El array proporcionado contiene también otros datos que utilizarás después, como el género y la fecha.

```
function Partida(juego, minutos, puntos = 0, ...etiquetas) {
  this.juego = juego;  // esta propiedad ya está resuelta
  // TODO 2A: guarda las otras tres propiedades en this.
  // El método sumarPuntos ya tiene su estructura preparada.
}
```

En el TODO 2A guarda los parámetros minutos, puntos y etiquetas como propiedades del objeto. Aquí solo debes guardar los datos, sin validarlos. ...etiquetas reúne los argumentos restantes en un array: puede haber cero, una o varias etiquetas.

En el TODO 2B completa el método sumarPuntos. La validación ya está escrita: solo queda sumar extra a this.puntos y devolver true. El parámetro extra tiene valor por defecto 100.

```
const p = new Partida("Galaxia", 15, 300, "solo", "reto");
console.log(p.etiquetas);  // ["solo", "reto"]
p.sumarPuntos();          // suma 100
p.sumarPuntos(50);        // suma otros 50
console.log(p.puntos);    // 450
```

Comprueba también estas llamadas:

Llamada	puntos guardados
new Partida("Galaxia", 15)	0
new Partida("Galaxia", 15, undefined)	0
new Partida("Galaxia", 15, null)	null

El valor por defecto se aplica al omitir el argumento o pasar undefined; null se conserva. Prueba además pasar un array de etiquetas con spread en la llamada: new Partida("Galaxia", 15, 0, ...["solo", "reto"]). En la declaración, rest las reúne; en la llamada, spread las separa en argumentos.

3 Copiar sin cambiar el original

Completa el TODO 3 de prepararCopias(partida, lista). Dentro de la función declara tres constantes con estos nombres exactos. El return que reúne los resultados ya está escrito.

Constante	Qué debes hacer
copiaSpread	Crear un objeto con los datos de partida y genero igual a "exploracion", usando spread de objeto.
copiaAssign	Obtener el mismo resultado con Object.assign y un objeto destino nuevo.
copiaArray	Copiar lista en un array nuevo usando spread de array.

Una propiedad escrita después de ...partida sustituye a la anterior con el mismo nombre. En Object.assign se modifica el primer argumento; empieza con {} para que el objeto de partida conserve sus datos.

```
const r = prepararCopias(partidas[0], partidas);
console.log(partidas[0].genero); // "aventura"
console.log(r.copiaSpread.genero); // "exploracion"
console.log(r.copiaArray !== partidas); // true
```

Son copias superficiales: el array nuevo sigue conteniendo las mismas partidas y los objetos copiados comparten el array etiquetas con el original. No necesitas copiar también ese array. Compruébalo con r.copiaSpread.etiquetas === partidas[0].etiquetas: debe dar true.

4 Trabajar con fechas

El conversor aTimestamp ya está resuelto. Devuelve una marca de tiempo en milisegundos o null si la entrada no es válida. En las funciones que debes completar también están preparadas las comprobaciones de error.

En el TODO 4A de describirFecha devuelve un objeto con las propiedades timestamp, anio, mes y dia, usando los métodos de fecha de esta tabla.

Dato	Cómo obtenerlo a partir de fecha
timestamp	fecha.getTime()
anio	fecha.getUTCFullYear()
mes	fecha.getUTCMonth() + 1
dia	fecha.getUTCDate()

En el TODO 4B de diasEntre ya tienes a y b, las marcas de inicio y fin. Devuelve (b − a) dividido entre 86400000, los milisegundos de un día de 24 horas. Conserva el signo y no redondees.

```
describirFecha("2026-10-18");
// { timestamp: 1792281600000, anio: 2026, mes: 10, dia: 18 }
diasEntre("2026-10-01", "2026-10-18"); // 17
console.log(typeof Date.now());       // "number"
```

Date.now() devuelve el timestamp actual. Usamos UTC para que los resultados no dependan de la zona horaria del ordenador. Recuerda que getUTCMonth() cuenta desde 0 y getUTCDate() da el día del mes, no el día de la semana.

5 Elegir partidas con filter

Completa el TODO 5 de filtrarPartidas. El recorrido con filter ya está escrito: dentro del callback debes devolver una condición que indique si conservar la partida.

```
function filtrarPartidas(lista, genero, minimo = 0) {
  return lista.filter(partida => {
    // TODO 5: devuelve la condición de selección.
  });
}
```

La duración debe ser mayor o igual que minimo. Además, si se ha recibido genero, tiene que coincidir exactamente con partida.genero. Si genero se omite o es undefined, no debe restringir la selección. Une las dos condiciones con && en la condición del género necesitarás ||.

Llamada	id de las partidas seleccionadas
filtrarPartidas(partidas, "carreras")	3, 7
filtrarPartidas(partidas, "carreras", 35)	7
filtrarPartidas(partidas, undefined, 40)	1, 4, 6
filtrarPartidas(partidas)	Todos, en un array nuevo

genero es opcional porque tu condición permite que falte, aunque no tenga un valor por defecto escrito en la cabecera. minimo sí tiene valor por defecto. Si se pasa null como género, se busca ese valor exacto y, con estos datos, el resultado es [].

6 Sumar y agrupar con reduce

Completa el TODO 6A de totalPuntos. El acumulador empieza en 0. En cada vuelta devuelve el total anterior más los puntos de la partida actual.

```
totalPuntos(partidas); // 7950
totalPuntos([]);       // 0
```

En las primeras vueltas, el acumulador pasa de 0 a 1200, después a 2000 y después a 2600. Si obtienes undefined, revisa que el callback devuelva el valor para la siguiente vuelta.

Completa el TODO 6B de puntosPorJuego. Aquí el acumulador es un objeto {} y el nombre del juego es la clave. La inicialización de cada clave ya está resuelta. Añade los puntos en acumulador[partida.juego] y conserva el return acumulador que aparece debajo.

```
puntosPorJuego(partidas);
// { "Órbita Azul": 3000, "Ciudad Pixel": 800,
//   "Rally Solar": 1700, "Bosque Run": 950, "Laboratorio X": 1500 }
```

Puedes modificar el objeto acumulador que acabas de crear; no modifiques las partidas. Al filtrar y reducir utiliza los métodos pedidos, sin sustituirlos por for, while o forEach.

Revisión y entrega

Reserva los últimos 15 minutos para ejecutar el archivo completo y corregir los casos pendientes o con fallo. La demostración final ya está escrita y combina tus filtros y sumas: debe mostrar dos sesiones de aventura y 3000 puntos. Entrega el archivo .js con tus cambios y el comentario del apartado 1.